

OpenCode Agent Prompt — RehabTrack, a Cardiac Rehab Management App (Flutter, Android)

Copy everything below the line into OpenCode.

This is a Flutter/Dart alternative to a parallel Qt/QML version of the same app. Data model and required screens are identical between the two — only the tech stack, architecture, and platform-integration approach differ.

Project Goal

Build **RehabTrack**, a personal Android application, using **Flutter (Dart)**, to help a single patient manage post-stent cardiac rehabilitation: therapy courses, medication prescriptions with flexible dosing schedules, dose reminders via real Android system notifications, dose adherence logging, and periodic health measurements (blood pressure, glucose, pulse, etc.) with reminders and history.

This is a **single-patient, local-only, offline-first** app. No cloud sync, no multi-user support, no server backend.

Tech Stack

- **Flutter (stable channel)**, Dart, Android as the only target platform for now (project structure may stay technically cross-platform, but no iOS/desktop work is required)
- **State management**: Riverpod (preferred) or Provider — pick one and use it consistently across the app; avoid mixing state-management approaches
- **Local persistence**: `drift` (preferred, type-safe SQL over SQLite with built-in migration support) or `sqflite` directly if the agent judges drift's codegen overhead isn't worth it for this project's size
- **Notifications & scheduling**: `flutter_local_notifications` for scheduling and displaying local notifications with action buttons, backed by Android's `AlarmManager` under the hood; combine with `android_alarm_manager_plus` or the notification plugin's own exact-alarm scheduling if finer control over exact/idle-tolerant alarms is needed
- **Boot rescheduling**: ensure `flutter_local_notifications`' Android receiver handling covers `BOOT_COMPLETED` rescheduling of pending alarms (verify plugin version supports this; otherwise add a thin native `BroadcastReceiver` via platform channel)
- **Biometric/PIN lock**: `local_auth` for biometric prompts; store a hashed PIN locally (e.g. via `crypto` package) for the PIN fallback — never store PIN in plaintext

- **File/photo picking:** `image_picker` (camera + gallery) and `file_picker` (for picking existing PDFs) for attaching lab analysis result files
- **PDF viewing:** `syncfusion_flutter_pdfviewer` or `flutter_pdfview` for in-app viewing of attached PDF lab results
- **PDF generation:** `pdf` + `printing` packages for generating the shareable report PDF described under "Share Report" below
- **CSV generation:** `csv` package for writing dose-log/measurement CSV exports
- **Sharing / email handoff:** `share_plus` for handing generated files (PDF/CSV) and text (shopping list) off to the device's share sheet / email app via `Intent.ACTION_SEND` under the hood — the app never sends email itself
- **Phone dialer handoff:** `url_launcher` with a `tel:` URI (equivalent to `Intent.ACTION_DIAL` — opens the dialer with the number pre-filled, does not call automatically, no special permission required)
- **Charts:** `fl_chart` or `syncfusion_flutter_charts` for measurement trend charts
- **Permissions:** `permission_handler` for requesting `POST_NOTIFICATIONS` (Android 13+), exact-alarm permission, and camera/storage permissions as needed
- **Battery optimization exemption:** platform channel call to request `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`, or a plugin such as `disable_battery_optimization` if it reliably covers this on current Android versions — verify before relying on it
- **File storage paths:** `path_provider` for locating app-private storage for the SQLite DB and lab-analysis files
- **Localization:** `flutter_localizations` (from the Flutter SDK) + the `intl` package, with ARB files (`app_en.arb`, `app_ka.arb`, etc.) as the translation source — see "Language Settings" screen below

Required Android manifest permissions (declared via

`android/app/src/main/AndroidManifest.xml`, some auto-added by the plugins above but verify explicitly):

- `RECEIVE_BOOT_COMPLETED`
- `SCHEDULE_EXACT_ALARM` (and `USE_EXACT_ALARM` fallback per Android 12+/13+ rules)
- `POST_NOTIFICATIONS` (Android 13+ runtime permission)
- `VIBRATE`
- `FOREGROUND_SERVICE` if a foreground service ends up being used alongside AlarmManager-based scheduling

Data Model (SQLite schema)

Identical schema to the Qt/QML version of this app — implement as `drift` tables (or raw `sqflite` CREATE TABLE statements) rather than SQL files, but keep the same

table/column names and relationships so a future migration between the two implementations, or a shared understanding of the data, stays straightforward. Use `INTEGER PRIMARY KEY AUTOINCREMENT` (or drift's `IntColumn().autoIncrement()`) for ids, and store dates/datetimes as ISO-8601 `TEXT` (or drift's `DateTimeColumn`), but be explicit about storing as UTC vs local wall-clock time given the timezone/DST note below).

`patient` (single row, enforced by app logic, not multi-row)

- `id`, `first_name`, `last_name`, `personal_id`, `birth_date`, `gender`, `height_cm`, `weight_kg`, `blood_type`, `allergies` (text), `emergency_contact_name`, `emergency_contact_phone`, `cardiologist_name`, `cardiologist_phone`, `notes`

`therapy_course`

- `id`, `name`, `duration`, `begin_date`, `end_date`, `status` (`ongoing` | `completed` | `stopped`), `is_archived` (bool, default false), `notes`

Deleting a therapy course from the UI must be a **soft delete**: set `is_archived = true` rather than removing the row. Archived courses (and everything linked to them — prescriptions, dose logs, measurement schedules/readings, lab analyses, visits) drop out of the normal "active" lists but remain fully queryable in an "Archived" view, since medical history should never be silently destroyed. Only allow a genuine hard delete for a course that has zero linked history (created by mistake, nothing logged against it yet) — gate this behind an explicit confirmation that makes clear it's permanent.

`prescription`

- `id`, `therapy_course_id` (FK), `notes`
- (one-to-one with `therapy_course`: a therapy course has exactly one prescription container holding many entries)

`prescription_entry`

- `id`, `prescription_id` (FK), `drug_name`, `dose_amount`, `dose_unit` (e.g. mg, ml, tablet), `schedule_type` (`fixed_times_per_day` | `interval_days`), `schedule_config` (JSON text — see below), `start_date`, `end_date` (nullable = ongoing/indefinite), `instructions` (free text, e.g. "take with food"), `active` (bool)

`schedule_config` JSON examples:

```
json
```

```
// 3 times a day at specific times
{ "type": "fixed_times_per_day", "times": ["08:00", "14:00", "20:00"] }

// once every 3 days at 09:00, starting from start_date
{ "type": "interval_days", "interval": 3, "time": "09:00" }

// twice, every other day
{ "type": "interval_days", "interval": 2, "times": ["08:00", "20:00"] }
```

Design the Dart model layer to parse/validate this generically (e.g. a sealed class / frozen union type per schedule type) so new schedule shapes can be added later without a schema migration.

dose_log

- id, prescription_entry_id (FK), scheduled_datetime, actual_datetime (nullable), status ((pending) | (taken) | (missed) | (skipped)), notified_at (nullable, for debugging missed notification delivery)

measurement_schedule

- id, therapy_course_id (FK, nullable — allow standalone schedules not tied to a course), measurement_type ((blood_pressure) | (glucose) | (pulse) | (weight) | (custom:<label>)), schedule_config (same JSON pattern as prescription_entry), active (bool), start_date, end_date (nullable)

measurement_reading

- id, measurement_schedule_id (FK, nullable if ad-hoc reading not tied to a schedule), measurement_type, timestamp, value_primary (e.g. systolic, or glucose value), value_secondary (nullable, e.g. diastolic), value_tertiary (nullable, e.g. pulse alongside BP), unit, notes

lab_analysis

- id, therapy_course_id (FK, nullable — allow standalone analyses not tied to a course), analysis_name (e.g. "Lipid panel", "INR", "Troponin"), date_taken, lab_or_facility (nullable free text), notes
- (one analysis record can have one or more attached result files — see (lab_analysis_file) below)

lab_analysis_file

- id, lab_analysis_id (FK), file_name, mime_type ((image/jpeg) | (image/png) | (application/pdf)), stored_path (path inside app-private storage, e.g. (<app documents dir>/lab_analyses/<uuid>.pdf)), file_size_bytes, added_at

Store the actual files in app-private storage (via `path_provider`'s application documents/support directory), never in the public Downloads folder by default — only copy a file there explicitly when the user chooses to download/export it, using Android's `MediaStore` via a platform channel or a plugin such as `saf_util`/`gallery_saver` depending on file type (no extra storage permission needed on modern Android for that path).

`doctor`

- id, first_name, last_name, phone, email (nullable), field (specialty — e.g. `cardiologist`, `surgeon`, `general_practitioner`, `other:<label>`; treat as a free-editable list of options rather than a hardcoded enum so the user can add new specialties), clinic_or_facility (nullable), notes

`doctor_visit`

- id, doctor_id (FK), therapy_course_id (FK, nullable — allow standalone visits not tied to a course), scheduled_datetime, status (`scheduled` | `completed` | `cancelled`), reason (free text, e.g. "follow-up", "post-op check"), notes, reminder_enabled (bool), reminder_lead_minutes (nullable — e.g. 1440 for "1 day before", 60 for "1 hour before"; null = remind at the exact scheduled time)

Doctor visits use the same notification-scheduling path as dose/measurement reminders (a single scheduled datetime rather than a recurring `schedule_config`), so a local notification fires ahead of the appointment (offer a configurable lead time, e.g. "remind me 1 day before" and/or "remind me on the day").

`diet_plan`

- id, therapy_course_id (FK, nullable — allow a standalone diet plan not tied to a course), title (e.g. "Post-stent low-sodium diet"), description (long free text — the doctor's overall guidance, in the patient's own words or copied from a handout), begin_date, end_date (nullable = ongoing/indefinite), status (`active` | `completed` | `stopped`), notes

`diet_guideline_item`

- id, diet_plan_id (FK), category (`recommended` | `avoid` | `limit`), item_text (short line, e.g. "Reduce sodium to under 2g/day", "Avoid fried and processed foods", "Leafy greens, oats, fatty fish"), notes (nullable, e.g. a reason or serving-size detail)

Structuring guidance into short `recommended`/`avoid`/`limit` line items (rather than one long paragraph) is what makes the "quickly see what I must follow" requirement actually usable day-to-day — the `description` field on `diet_plan` remains available for the fuller doctor's-note version alongside it.

`diet_plan_file`

- `id`, `diet_plan_id` (FK), `file_name`, `mime_type` (`image/jpeg` | `image/png` | `application/pdf`), `stored_path` (app-private storage path), `file_size_bytes`, `added_at`

Same attachment pattern as `lab_analysis_file` — lets the user photograph or attach a PDF of a printed diet sheet handed to them by the doctor, rather than having to retype everything as structured guideline items (though doing both — a quick photo plus a few key structured items — is the intended common case).

`app_setting`

- `key` (TEXT, primary key), `value` (TEXT)

Simple key-value table for global preferences that don't warrant their own table, e.g.:

- `ui_language` = BCP-47 language tag, e.g. `en`, `ka` (Georgian) — see "Language Settings" screen below
- `unit.glucose` = `mg/dL` or `mmol/L`
- `unit.weight` = `kg` or `lb`
- `app_lock.enabled` = `true`/`false`
- `app_lock.method` = `pin` or `biometric`
- `app_lock.pin_hash` (nullable — hashed, never store the PIN in plaintext)

Measurement types with region-dependent units (glucose especially: mg/dL vs mmol/L) must read their unit from `app_setting` rather than hardcoding one, and the measurement entry/history UI should label values with whichever unit is configured. Changing the unit setting later should not silently reinterpret old readings — store the unit alongside each `measurement_reading` row (already in that table) so historical data stays correctly labeled even if the user changes the global default afterward.

Application Architecture

- **Layered structure:** `data/` (drift tables + DAOs, or sqflite repositories), `domain/` (plain Dart model classes, schedule-parsing logic), `presentation/` (screens + widgets), `providers/` or `state/` (Riverpod providers or Provider/ChangeNotifier classes) — keep SQL/DB access out of widget code entirely.
- **Repository pattern:** one repository class per table/aggregate (`PatientRepository`, `TherapyCourseRepository`, `PrescriptionRepository`, `DoseLogRepository`, `MeasurementRepository`, `LabAnalysisRepository`, `DoctorRepository`, `DoctorVisitRepository`), each exposing typed methods (no raw SQL leaking into the UI layer).

- **NotificationScheduler service:** a single Dart class responsible for computing next-fire datetimes from `(schedule_config)`, calling into `(flutter_local_notifications)` to schedule/cancel Android alarms, and re-syncing all active schedules on app start (and relying on the plugin's boot-rescheduling support, verified during implementation).
 - **Database migrations:** use drift's built-in schema versioning and migration steps (or a manual `(schema_version)` table + migration runner if using raw sqflite) so future changes don't require reinstall/data loss.
 - **Material 3 theming** (`(useMaterial3: true)`) for a native Android look and feel out of the box.
-

Required Screens / Features

1. **Today Dashboard (home screen)** — the screen the app opens to. A single at-a-glance view of: doses due today (with quick "Taken"/"Skip" actions inline, no need to drill into a course), measurements due today, any upcoming doctor visit within the next few days, and a quick-glance summary of the active diet's key `(recommended)`/`(avoid)` items. This is the primary daily-use screen; everything else is reached from here (bottom navigation bar or drawer).
2. **Patient Info** — single form, create/edit patient details listed above.
3. **Therapy Course List** — list of courses with name, date range, status badge (color-coded ongoing/completed/stopped). Add/edit/delete. Filter by status.
4. **Therapy Course Detail** — tabs or sections for:
 - Course metadata (name, dates, status, notes)
 - **Prescription** — list of prescription entries with drug name, dose, schedule summary (human-readable, e.g. "3x daily at 08:00, 14:00, 20:00" or "every 3 days at 09:00"), add/edit/delete entries. Schedule editor UI: toggle between "times per day" (chip list of times, add/remove) and "every N days" (interval spinner + time picker) — must be simple, not a raw JSON editor.
 - **Measurement schedules** — list of measurement schedules for this course (type, frequency, active toggle switch), add/edit/delete.
5. **Prescription Shopping List** — aggregated, printable/shareable view across all active prescription entries (across all ongoing courses): drug name, dose, and computed quantity needed (e.g. based on frequency × remaining days until end_date, if end_date is set). Share via `(share_plus)` (plain text share sheet).
6. **Dose Reminders / Notifications** — local notification fires at scheduled time with drug name + dose, and action buttons: "Taken", "Snooze 15m", "Skip" (via `(flutter_local_notifications)` notification actions). Tapping an action writes to `(dose_log)` through a background callback and, for "Taken", also reflects in the app if it's open.
7. **Dose Adherence Log** — chronological or calendar view of `(dose_log)` entries per drug or per course, showing taken/missed/skipped, with simple adherence % stat.

8. **Measurement Entry** — quick-entry form triggered either from a measurement reminder notification or manually from the app (type-specific fields: BP = systolic/diastolic/optional pulse; glucose = single value + unit; weight = single value; custom = single labeled value).
9. **Measurement History** — list and simple trend chart (`fl_chart` or `syncfusion_flutter_charts`) per measurement type, filterable by date range, viewable on demand.
10. **Notification Settings** — global toggle, permission-request flow for `POST_NOTIFICATIONS` (Android 13+) and exact-alarm permission (Android 12+), snooze duration default.
11. **Lab Analyses / Test Results** — list of analyses (name, date taken, course if linked), sorted newest first, with a thumbnail/icon per attached file. Features:
 - **Add** — form for analysis name, date taken, facility/notes, plus one or more file attachments via `image_picker` (camera capture for a quick photo of a paper result, or gallery) and `file_picker` (for existing PDFs).
 - **View** — tapping an entry opens its file(s) in-app: render images directly, and for PDFs use `syncfusion_flutter_pdfviewer`/`flutter_pdfview`.
 - **Download** — a "save to device" action per file that copies it out of app-private storage into the public Downloads folder, so the user can access it outside the app (e.g. to email to a doctor).
 - **Delete** — remove an analysis and its attached files (with confirmation, since this deletes the underlying file too).
12. **Doctors** — simple directory: list of doctors with name, specialty/field, phone, email, clinic; add/edit/delete. A "Call" button on each doctor uses `url_launcher` with a `tel:` URI, opening the system's default phone app with the number pre-filled — the app never places the call itself, and the user confirms and places the call from there. This requires no special Android permission.
13. **Doctor Visits / Appointments** — list of upcoming and past visits (doctor name, specialty, datetime, status, linked course if any). Add/edit a visit (pick doctor, datetime, optional linked therapy course, reason, reminder lead time). Upcoming visits trigger a local notification reminder; mark a visit `completed` or `cancelled` after the fact.
14. **Missed-Item Catch-Up** — not a standalone screen so much as app-open behavior: on every app launch (and optionally surfaced as a banner on the Today Dashboard), check for `dose_log` rows still `pending` whose `scheduled_datetime` has passed, and equivalent overdue measurement schedules, and present them as "you have N missed items" with the ability to mark each taken/skipped retroactively. This covers cases where a notification never fired (phone off, Do Not Disturb, alarm killed by the OS) so nothing silently falls through as neither taken nor logged missed.
15. **Archived Courses** — a separate list (reached from Therapy Course List, e.g. via a filter/tab) showing soft-deleted/archived courses in read-only-ish form, with their full history intact and an option to unarchive.

16. **App Lock (optional, off by default)** — a setting under Notification Settings (or its own "Security" section) letting the user turn on a PIN or biometric (fingerprint/face, via `local_auth`) lock on app open. Off by default so it doesn't add friction for someone who doesn't want it; the PIN, if used, is stored hashed, never in plaintext.
17. **Share Report (email)** — reachable both as a standalone "Share Report" entry point and as a "Send to doctor" action from the Dose Adherence Log and Measurement History screens. Lets the user:
 - Pick a **date range** (e.g. "last 7 days", "last 30 days", "this therapy course", custom range).
 - Pick **what to include**: dose adherence log, measurement readings (per type), or both.
 - Pick a **format**: **PDF** (readable report, good for a doctor to glance at, generated via the `pdf`/`printing` packages) or **CSV** (raw tabular data via the `csv` package — one file for dose log entries, one for measurement readings, or a combined zip if both are selected — good for opening in Excel/Sheets for manipulation or long-term analysis).
 - Generate the chosen file(s) into app-private storage, then include a header row and ISO-8601 datetimes for CSV, one row per dose or reading.
 - Choose a **recipient**: pick from the Doctors directory (pre-fills their email if one is on file) or type an arbitrary email address freehand.
 - Hand the generated file(s) off to the device's email app via `share_plus`'s `Share.shareXFiles` (with subject/body pre-filled where the share sheet supports it, e.g. "Rehab report — [patient name] — [date range]"), so the user reviews and taps send themselves in their own mail client — no email credentials or SMTP setup inside the app itself.
18. **Language Settings** — a setting screen (e.g. under a general "Settings" area alongside Notification Settings and App Lock) letting the user pick the interface language from a list, applied immediately via a locale `StateNotifier`/`ChangeNotifier` without requiring an app restart. Start with **English and Georgian** as the two supported languages (matching the developer's own usage), structured so additional languages are just another ARB file, not a code change. Every user-facing string must go through the generated `AppLocalizations` class (via `flutter_gen-l10n`) from the start — do not hardcode any UI text, including button labels, notification text, validation messages, and the schedule-summary strings (e.g. "3x daily at 08:00, 14:00, 20:00" must itself be built from translatable pieces via ICU message syntax, not a hardcoded English sentence template).
19. **Diet Plan** — a dedicated read-focused view answering "what diet must I follow right now." Shows the current `active` diet plan (or plans, if more than one is active) at a glance:
 - Three clearly separated lists — **Recommended, Avoid, Limit** — built from `diet_guideline_item` rows, since scanning short bullet lines is far faster in the moment (e.g. standing in a grocery aisle) than reading a paragraph.

- The full free-text `(description)` available below/behind the lists for the complete doctor's guidance.
 - Any attached `(diet_plan_file)` photos/PDFs of a printed diet sheet, viewable the same way as lab analysis files.
 - **Add/edit** — create a diet plan (title, dates, status, description), add guideline items under each category, attach files.
 - Past (non-active) diet plans are reachable from a simple history list, same soft-visibility pattern as Archived Courses — nothing is hard-deleted, since a past diet plan is still useful medical history.
-

Non-Functional Requirements

- **Offline-first, fully local** — no network permissions requested unless explicitly added later for backup/export.
- **Data export** — provide a "backup" feature that exports **everything** to a single `(.zip)` file (via the `(archive)` package): a JSON dump of all database tables plus the actual contents of the lab-analysis files folder (images/PDFs), so a restore recovers the full picture, not just metadata pointing at files that may no longer exist. Provide a matching "restore from backup" import flow that validates the zip structure before overwriting local data (with a confirmation step, since restore is destructive to current data).
- **Robustness of reminders** — verify exactly how the chosen notification plugin handles Doze mode / exact alarm restrictions and boot-time rescheduling before relying on it; if gaps exist, close them with a thin native Android `(BroadcastReceiver)` via a platform channel. On first run, also prompt the user to exempt the app from battery optimization `(REQUEST_IGNORE_BATTERY_OPTIMIZATIONS)` with a short explanation of why, since OEM battery managers (Xiaomi, Huawei, Samsung, etc.) are known to kill scheduled alarms otherwise.
- **Timezone/DST safety** — store scheduled times as local wall-clock time + recompute next-fire on each trigger, not as fixed absolute timestamps, so DST transitions don't shift dose times.
- **Privacy** — this is personal medical data; avoid logging PII via `(print)`/`(debugPrint)` in release builds. Consider `(sqlcipher_flutter_libs)` (encrypted SQLite) as a stretch goal if the agent has time, otherwise note it as a follow-up.
- **Validation** — enforce sensible constraints (`end_date ≥ begin_date`, `dose amount > 0`, at least one time in a schedule, etc.) with inline form validation `(Form)`/`(TextFormField)` validators) and user-facing error messages.
- **Localization** — treat multi-language support as a first-class constraint from the start, not an afterthought: all user-facing strings (UI labels, notification text, validation errors, share/report text) must be translatable via `(AppLocalizations)`, dates/times formatted per-locale `(intl)`'s `(DateFormat)`, and adding a new language later should

require only a new ARB file, no code changes. Note that notification text scheduled via `flutter_local_notifications` is generated at schedule-time, not display-time — regenerate/reschedule pending notification text if the user changes the language setting, so notifications don't show stale-language text after a language switch.

Deliverables Expected

- Full Flutter project structure (`pubspec.yaml`) with all dependencies pinned, `android/` manifest configured with the permissions above)
- Database schema (drift schema files or sqflite CREATE TABLE statements) and a migration mechanism
- Dart repository/model/provider classes as described above
- Flutter UI (widgets/screens) for all screens listed
- README with build/run instructions for Android deployment, and a summary of the notification/permission flow the user will need to grant on first run

Start by proposing the project folder structure and the database schema (with migration versioning in place), then proceed incrementally screen by screen, confirming the data layer compiles and the schema is sound before building UI on top of it.

Building & Distributing a Release APK

The app will be distributed by directly sharing a signed release APK file (e.g. via Google Drive link, email, or USB), not through the Play Store initially. Set this up properly from the start so it isn't a scramble later:

- **Generate a release keystore** (`keytool -genkey -v -keystore rehabtrack-release.jks -keyalg RSA -keysize 2048 -validity 10000 -alias rehabtrack`) and document the exact command used in the README. This keystore must be kept safe and reused for every future release — losing it means future updates can't be installed over old ones (Android treats differently-signed APKs as different apps).
- **Configure release signing** in `android/app/build.gradle`, referencing the keystore via a local, git-ignored `android/key.properties` file (Flutter's standard pattern) — never commit signing credentials to source control.
- **Set a sensible `applicationId`** in `android/app/build.gradle` (e.g. `com.<developer>.rehabtrack`) and human-readable app name/icon before the first release, since changing the `applicationId` later effectively makes it a different app for update purposes.
- **Version the app** — bump `version` in `pubspec.yaml` (`versionName+versionCode` format, e.g. `1.0.0+1`) on each release, and mention the current version in the README.

- **Build command:** `flutter build apk --release` (document this and where the signed APK lands, typically `build/app/outputs/flutter-apk/app-release.apk`).
- **README distribution notes:** include a short, non-technical paragraph the developer can adapt when sending the APK to another patient — explaining that they'll need to enable "install unknown apps" for whichever app they download the APK through, since it isn't from the Play Store, and that this is expected and safe for an app shared directly by a trusted source.